

We can formalize this insight a bit. Given an impure function f of type $A \Rightarrow B$, we can split f into two functions:

- A *pure* function of type $A \Rightarrow D$, where D is some *description* of the result of f .
- An *impure* function of type $D \Rightarrow B$, which can be thought of as an *interpreter* of these descriptions.

We'll extend this to handle "input" effects shortly. For now, let's consider applying this strategy repeatedly to a program. Each time we apply it, we make more functions pure and push side effects to the outer layers. We could call these impure functions the "imperative shell" around the pure "core" of the program. Eventually, we reach functions that seem to necessitate side effects like the built-in `println`, which has type `String => Unit`. What do we do then?

13.2 A simple IO type

It turns out that even procedures like `println` are doing more than one thing. And they can be factored in much the same way, by introducing a new data type that we'll call `IO`:

```
trait IO { def run: Unit }

def PrintLine(msg: String): IO =
  new IO { def run = println(msg) }

def contest(p1: Player, p2: Player): IO =
  PrintLine(winnerMsg(winner(p1, p2)))
```

Our `contest` function is now pure—it returns an `IO` value, which simply describes an action that needs to take place, but doesn't actually execute it. We say that `contest` has (or produces) an *effect* or is *effectful*, but it's only the interpreter of `IO` (its `run` method) that actually has a *side effect*. Now `contest` only has one responsibility, which is to compose the parts of the program together: `winner` to compute who the winner is, `winnerMsg` to compute what the resulting message should be, and `PrintLine` to indicate that the message should be printed to the console. But the responsibility of *interpreting* the effect and actually manipulating the console is held by the `run` method on `IO`.

Other than technically satisfying the requirements of referential transparency, has the `IO` type actually bought us anything? That's a personal value judgement. As with any other data type, we can assess the merits of `IO` by considering what sort of algebra it provides—is it something interesting, from which we can define a large number of useful operations and programs, with nice laws that give us the ability to reason about what these larger programs will do? Not really. Let's look at the operations we can define:

```
trait IO { self =>
  def run: Unit
  def ++(io: IO): IO = new IO {
    def run = { self.run; io.run } }
```

← The `self` argument lets us refer to this object as `self` instead of `this`.

← `self` refers to the outer `IO`.